

SOAP Engine

Die SOAP-Engine ermöglicht die Ausführung von SOAP-basierten Web-Service-Aufrufen (XML-Schnittstellen über HTTP POST) innerhalb von Testschritten. Die Engine unterstützt die dynamische URL- und Body-Generierung mittels Platzhaltern, bietet automatische Mechanismen zur Bereinigung von XML-Payloads (Entfernung von leeren oder `{NULL}` - Knoten), decodiert komprimierte GZIP-Antworten, isoliert SOAP-Envelopes aus MTOM/Multipart-Antworten, fügt vollautomatisch WS-Security-Header ein, unterstützt **bidirektionales MTOM/INLINE-Handling (Upload & Download)** und validiert bzw. extrahiert Daten über standardisierte XPath-Ausdrücke.

1. SOAP-Engine – Kompakte System-Referenz für KI-Modelle

```
1 | YAML
1 # AGATE SOAP-Engine Syntax Referenz
2 # Für die Generierung von Testschritten und Analysen verwenden
3
4 - type: SOAP
5   op: EXEC
6   command: soap.countryinfo.ListOfCountryNamesGroupedByContinent
7   endpoint: "http://webservices.oorsprong.org"
8   response: response
9   condition: "..." # Optional: '{B[var]}' == 'val' (einfache Anf
10  parameters:
11    dialogId: "..."
12  upload: # Optional: Liste von ausgehenden Dokumenten
13    - method: MTOM|INLINE
14      path: "//*[local-name()='attachment']" # XPath-Zielknoten im Requ
15      sourceFile: "attachement.abs.dokument.zip" # Relativer Pfad im Do
16  download: # Optional: Liste von eingehenden Dokumenten
17    - method: MTOM|INLINE
18      path: "//*[local-name()='druckaufbereitung']/text()" # XP
19      targetPath: "target/downloads/datenblatt_{B[svNummer]}.pdf" # Da
20
21 - type: SOAP
22   op: ASSERT
23   response: "soap_response" # Schlüssel der gespeicherten Antwort
24   source: "STATUS|BODY|HEADERS" # Validierungsquelle
25   path: "//*[local-name()='Status']/text()" # XPath für BODY, Header-Na
26   action: "EQUALS|NOT_EQUALS|CONTAINS|IS_EMPTY|IS_NOT_EMPTY" # Vergleic
27   expected: "OK" # Erwarteter Wert (unterstützt {NULL} un
28
29 - type: SOAP
30   op: BUFFER
31   response: "soap_response" # Quelle, die gelesen wird
32   source: "BODY"
33   path: "//*[local-name()='Quittung']/*[local-name()='Id']/text()" # Zi
34   name: "quittung_id" # Name der Zielvariable im Kontext
35   constraints: # Optional: Filterung bei Listen-Element
36     - path: "//*[local-name()='Quittung']/*[local-name()='Type']"
37       action: "EQUALS|NOT_EQUALS|CONTAINS|IS_EMPTY|IS_NOT_EMPTY" # Verg
38       expected: "REGULAR"
39
```

2. Operationen (DSL op)

Operation	Beschreibung
EXEC	Erstellt und sendet eine HTTP-POST-Anfrage basierend auf den Konfigurationsdateien.
ASSERT	Validiert den HTTP-Statuscode, Header oder XML-Inhalte einer vorherigen SOAP-Antwort.
BUFFER	Extrahiert spezifische Datenwerte mittels XPath aus der Antwort in Kontext-Variablen.

2.1. EXEC (Kommando-Ausführung)

Erstellt und sendet eine HTTP-POST-Anfrage mit einem integrierten `HttpClient`. Die SSL-Verifizierung ist standardmäßig deaktiviert (selbstsignierte Zertifikate im Testumfeld werden akzeptiert). Der Verbindungs-Timeout beträgt 10 Sekunden, der Request-Timeout 15 Sekunden.

Parameter	Beschreibung	Erforderlich
command	Der logische Pfad zum Verzeichnis, das die <code>metadata.json</code> und <code>request.xml</code> enthält.	Ja
endpoint	Die Basis-URL des Ziel-Webservices (ersetzt <code>{{endpoint}}</code> in der Metadaten-URL).	Ja
response	Der eindeutige Schlüssel, unter dem die Antwort im Kontext für nachfolgende Schritte gespeichert wird.	Ja

condition	Bedingung für die Ausführung des Schritts (Evaluierung vor dem Request).	Nein
parameters	Schlüssel-Wert-Paare zur dynamischen Ersetzung von Platzhaltern in der <code>request.xml</code> .	Nein
download	Liste von Objekten zur automatischen Dateixtraktion (<code>method</code> , <code>path</code> , <code>targetPath</code>).	Nein
upload	Liste von Objekten zur automatischen Dateiinjektion vor dem Senden (<code>method</code> , <code>path</code> , <code>sourceFile</code>).	Nein

Beispiel:

```

1  - type: SOAP
2    op: EXEC
3    command: soap.calculator.add
4    endpoint: "http://www.dneonline.com"
5    parameters:
6      intA: "{B[x]}"
7      intB: "{B[x]}"
8    response: response

```

Funktionsweise & Datei-Mapping

Der logische Pfad zur `metadata.json` und `request.xml` wird über den Parameter `command` definiert:

`metadata.json`

```

1  {
2    "url" : "{{endpoint}}/calculator.asmx",
3    "method" : "POST",
4    "headers" : {
5      "Content-Type" : "text/xml;charset=UTF-8",
6      "SOAPAction" : "\"http://tempuri.org/Add\"",
7      "User-Agent" : "Agate-HttpClient (Version 1.0.0)"

```

```
8   }
9 }
```

request.xml

```
1 <soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
2   xmlns:xsd="http://www.w3.org/2001/XMLSchema"
3   xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
4   <soap:Body>
5     <Add xmlns="http://tempuri.org/"
6       <intA>{B[intA]}</intA>
7       <intB>{B[intB]}</intB>
8     </Add>
9   </soap:Body>
10 </soap:Envelope>
```

metadata.json und request.xml liefern die parametrisierten Informationen, die notwendig sind, damit der AGATE HTTP-Client den SOAP-Request korrekt auflösen und senden kann.

Beispiel: Test-Schritt

```
1 - id: TC_SOAP_Calculator
2   description: "Public Service http://www.dneonline.com/calculator.as
3   stage: "*"
4   priority: HIGH
5   variables:
6     x: '15'
7     y: "5"
8
9   steps:
10
11     # 1.
12     - type: SOAP
13       op: EXEC
14       command: soap.calculator.add
15       endpoint: "http://www.dneonline.com"
16       parameters:
17         intA: "{B[x]}"
18         intB: "{B[y]}"
19       response: response
20
```

Konsolen-Output der Engine (Request auf Server)

```
1 >>> CALL      : POST http://www.dneonline.com/calculator.asmx
2 >>> HEADER     : Content-Type: text/xml;charset=UTF-8
3 >>> HEADER     : SOAPAction: "http://tempuri.org/Add"
4 >>> HEADER     : User-Agent: Agate-HttpClient (Version 1.0.0)
5 >>> BODY       : <soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLS
6 >>> BODY       :           xmlns:xsd="http://www.w3.org/2001/XMLS
7 >>> BODY       :           xmlns:soap="http://schemas.xmlsoap.org
8 >>> BODY       :     <soap:Body>
9 >>> BODY       :       <Add xmlns="http://tempuri.org/"
10 >>> BODY       :         <intA>15</intA>
11 >>> BODY       :         <intB>5</intB>
```

```

12 >>> BODY      :      </Add>
13 >>> BODY      :      </soap:Body>
14 >>> BODY      :      </soap:Envelope>

```

Konsolen-Output der Engine (**Response vom Server**)

```

1 <<< HEADER : cache-control: private, max-age=0
2 <<< HEADER : content-length: 326
3 <<< HEADER : content-type: text/xml; charset=utf-8
4 <<< HEADER : date: Fri, 26 Jun 2026 23:29:46 GMT
5 <<< HEADER : server: Microsoft-IIS/10.0
6 <<< HEADER : x-aspnet-version: 2.0.50727
7 <<< HEADER : x-powered-by: ASP.NET
8 <<< HEADER : x-powered-by-plesk: PleskWin
9 <<< BODY   : <soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/
10 <<< BODY   :      <soap:Body>
11 <<< BODY   :      <AddResponse xmlns="http://tempuri.org/">
12 <<< BODY   :      <AddResult>20</AddResult>
13 <<< BODY   :      </AddResponse>
14 <<< BODY   :      </soap:Body>
15 <<< BODY   :      </soap:Envelope>
16

```

Eingehende Datei-Extraktion: `download`

Falls der SOAP-Response binäre Daten (z. B. generierte PDFs, XML-Zertifikate) enthält, kann direkt innerhalb des `EXEC` -Schritts ein `download` -Block definiert werden. Die Engine extrahiert diese Daten automatisch, schreibt sie auf die Festplatte und optimiert den im Kontext gespeicherten XML-String.

```

1 | YAML
1 - type: SOAP
2   op: EXEC
3   command: soap.dbas.amp.13
4   endpoint: "https://services-t.ecard-test.sozialversicherung.at:1
5   download:
6     - method: MTOM
7       path: "//*[local-name()='druckaufbereitung']/text()"
8       targetPath: "target/downloads/datenblatt_{B[svNummer]}.pdf"
9

```

Parameter im `download` -Block:

- `method` (*Optional*): Der Extraktionsmechanismus.
 - `INLINE` : Die Datei liegt als Base64-kodierter Text direkt innerhalb eines XML-Tags (Standardwert).
 - `MTOM` : Reserviert für binäre MIME-Multipart-Anhänge (XOP Include).
- `path` (*Erforderlich*): Ein valider XPath-Ausdruck, der auf den Zielknoten im Response-XML zeigt.

- `targetPath` (*Erforderlich*): Der lokale Zielpfad inklusive Dateiname. Unterstützt die dynamische Variablenauflösung (z. B. `{B[svNummer]}`).

2.1.1. Erweiterte Engine-Features und Optimierungen

2.1.1.1. In-Place Base64-Dateiersetzung (RAM-Schonung)

Große Base64-Datenströme (z. B. mehrseitige PDFs) innerhalb von SOAP-Antworten können den JVM-Heap stark belasten und die System-Logs unleserlich machen. Um dies zu verhindern, wendet die Engine bei der Nutzung des `download`-Parameters eine **In-Place-Mutation** des XML-DOM-Baums an:

1. **Extraktion:** Der Base64-String wird über den XPath ausgelesen und als physische Binärdatei auf die Festplatte geschrieben.
2. **Mutation:** Der originale, massive Base64-Textinhalt wird innerhalb des betroffenen XML-Knotens gelöscht.
3. **Referenzierung:** Der Knoteninhalt wird durch eine kompakte Dateireferenz im Format `file:` `<targetPath>` ersetzt (z. B. `<druckaufbereitung>file:target/downloads/datenblatt_5248019250.pdf</druckaufbereitung>`).

Vorteil für nachfolgende Schritte: Da die Pfad im XML erhalten bleibt, können nachfolgende `ASSERT` - oder `BUFFER` -Schritte den Pfad mittels XPath auslesen oder validieren:YAML

```

1  - type: SOAP
2    op: ASSERT
3    response: datenblatt_res
4    source: BODY
5    path: "//*[local-name()='druckaufbereitung']/text()"
6    action: CONTAINS
7    expected: "file:target/downloads/"

```

Ausgehende Datei-Injektion: `upload`

Falls die Schnittstelle Binärdateien im Request erfordert, injiziert die Engine diese vor dem Senden über den `upload`-Block:

```

1  YAML
2
3  - type: SOAP
4    op: EXEC
5    command: soap.foo.anfrage.25
6    endpoint: "https://servicesco.at"
7    parameters:
8      dialogId: "{B[dialogId]}"
9    upload:
10     - method: MTOM
11       path: "//*[local-name()='attachment']"
12       sourceFile: "attachement.foo.dokument.zip"
13     response: datenblatt_res

```

- **Dot-Notation-Pfade:** Der Parameter `sourceFile` verwendet ein Punkttrennzeichen-System, das **relativ zum Verzeichnis des aktuellen YAML-Testfall-Fails** aufgelöst wird.
 - *Beispiel:* Liegt der Testfall unter `.../data/demo/soap_service.yaml` und die Datei unter `.../data/demo/attachement/foo/dokument.zip`, lautet die Dot-Notation: `attachement.abs.dokument.zip`.
- **Unterstützte Methoden:**
 - **INLINE** : Die Datei wird vollautomatisch in Base64 konvertiert und direkt als Text in das XML-Element injiziert.
 - **MTOM** : Der XML-Zielknoten wird geleert und mit einem standardkonformen `<xop:Include href="cid:..." />`-Element versehen. Die Datei wird als binärer MIME-Multipart-Anhang an den HTTP-Body angehängt, und der HTTP-Header **Content-Type** wird dynamisch auf `multipart/related` inklusive einer generierten Boundary umgestellt.

2.2. ASSERT (Validierung) für HEADERS, BODY und STATUS

Prüft den Inhalt oder den HTTP-Statuscode einer zuvor gespeicherten Antwort. Schlägt eine Prüfung fehl, bricht der Test sofort ab (**Fail-Fast-Prinzip**).

Unterstützte Aktionen für HEADERS und BODY

Aktion	Beschreibung	Erforderliche Parameter
CONTAINS	Prüft, ob der erwartete Text im extrahierten Wert enthalten ist.	<code>expected</code>
NOT_CONTAINS	Prüft, ob der erwartete Text im extrahierten Wert <i>nicht</i> enthalten ist.	<code>expected</code>
EQUALS	Prüft auf exakte Übereinstimmung (inklusive automatischem <code>trim()</code>).	<code>expected</code>
NOT_EQUALS	Prüft auf Abweichung vom erwarteten Wert.	<code>expected</code>

EMPTY	Prüft, ob das Feld leer oder der XPath-Knoten nicht vorhanden ist.	
NOT_EMPTY	Prüft, ob das Feld Daten enthält.	

Beispiele:

```

1  - type: SOAP
2    op: ASSERT
3    source: BODY
4    path: /*[local-name()='Envelope']/*[local-name()='Body']/*[local-name()='Status']
5    action: EQUALS
6    expected: 'false'
7    response: soap_response

```

Unterstützte Aktionen für STATUS (Numerisch)

Prüft den Inhalt oder Status eines zuvor gespeicherten Outputs. Schlägt eine Prüfung fehl, bricht der Test ab (**Fail-Fast**).

Aktion	Beschreibung	Erforderliche Parameter
EQUALS	Prüft auf exakten numerischen Status (z. B. 200).	expected
GREATER_THAN	Wert muss größer als der erwartete Wert sein.	expected
GREATER_OR_EQUALS	Wert muss größer oder gleich dem erwarteten Wert sein.	expected
LESS_THAN	Wert muss kleiner als der erwartete Wert sein.	expected

LESS_OR_EQUALS	Wert muss kleiner oder gleich dem erwarteten Wert sein.	expected
-----------------------	---	-----------------

Beispiele:

```

1 - type: SOAP
2   op: ASSERT
3   source: STATUS
4   action: EQUALS
5   expected: '200'
6   response: soap_response

```

2.2.1. ASSERT MATCH_REFERENCE – Validierung kompletter SOAP-Responses

Neben der gezielten Validierung einzelner Werte über XPath unterstützt die SOAP-Engine mit **MATCH_REFERENCE** auch den **semantischen Vergleich einer vollständigen SOAP-Response mit einer gespeicherten Referenzantwort**.

Diese Variante eignet sich insbesondere für Regressionstests, bei denen nicht nur einzelne Felder, sondern die komplette fachliche Struktur einer SOAP-Antwort überprüft werden soll.

Der Tester muss dabei nicht für jedes einzelne Response-Feld einen eigenen ASSERT-Schritt definieren.

Grundprinzip

Ein **MATCH_REFERENCE** -Assert wird wie ein normaler SOAP-ASSERT auf eine zuvor gespeicherte Response angewendet:

```

1 - id: verify_get_admin_response
2   type: SOAP
3   op: ASSERT
4   response: get_admin_response
5   source: BODY
6   action: MATCH_REFERENCE
7

```

Die Engine vergleicht den aktuellen XML-Body mit einer permanent gespeicherten XML-Referenz.

Der Ablauf unterscheidet zwischen dem **ersten Testlauf** und allen **nachfolgenden Testläufen**.

Erster Testlauf – automatische Erstellung der Referenz

Existiert für den Testschritt noch keine Referenzdatei, wird die aktuelle SOAP-Response automatisch als neue Referenz gespeichert.

Beispiel:

```
1 >>> SOAP ASSERT REFERENCE CREATED :
2   .../references/Matcher_dmp_11_getAdminPatientenInformationen/
3   TC001__verify_get_admin_response.xml
4
5 >>> REVIEW REQUIRED :
6   Check the generated reference response and keep/commit it only if
7   it is correct.
```

Der Test wird in diesem Fall **nicht als fehlgeschlagen bewertet**.

Die erzeugte Datei muss anschließend vom Tester fachlich überprüft werden.

Ist die Response korrekt, bleibt die Datei bestehen und kann zusammen mit den Testfällen in Git eingchecked werden.

Wichtig: Eine einmal vorhandene Referenzdatei wird von AGATE niemals automatisch überschrieben. Dadurch kann eine bereits geprüfte Referenz nicht versehentlich durch eine fehlerhafte Serverantwort ersetzt werden.

Nachfolgende Testläufe – Vergleich mit der Referenz

Existiert bereits eine Referenzdatei, vergleicht AGATE die aktuelle SOAP-Response mit dieser Referenz.

Sind beide Responses semantisch identisch, ist der ASSERT erfolgreich:

```
1 >>> SOAP ASSERT SUCCESS: MATCH_REFERENCE
2   Reference: .../TC001__verify_get_admin_response.xml
3
```

Treten Abweichungen auf, schlägt der ASSERT fehl.

Beispiel:

```
1 >>> SOAP ASSERT MATCH_REFERENCE: FAILED
2 >>> REFERENCE:
3   .../TC001__verify_get_admin_response.xml
4
5 >>> DIFFERENCE [1] VALUE_CHANGED
6   path      : /Envelope/Body/getAdminResponse/patient/name
7   expected  : Mustermann
8   actual    : Musterfrau
9
```

Dadurch ist unmittelbar erkennbar:

- an welcher Stelle die Response abweicht,
- welcher Wert in der Referenz erwartet wurde,
- welcher Wert tatsächlich vom Service geliefert wurde.

Der Test wird anschließend nach dem üblichen **Fail-Fast-Prinzip** beendet.

Semantischer XML-Vergleich

`MATCH_REFERENCE` führt keinen einfachen String-Vergleich der XML-Dateien durch.

Stattdessen werden die XML-Dokumente strukturell bzw. semantisch verglichen.

Unterschiede, die für die fachliche Aussage des XML-Dokuments keine Bedeutung haben, führen deshalb nicht automatisch zu einem Fehler.

Insbesondere werden beim Vergleich berücksichtigt:

- unterschiedliche XML-Formatierung und Einrückung,
- irrelevante Whitespaces,
- XML-Deklarationen,
- unterschiedliche Reihenfolge von XML-Attributen,
- unterschiedliche Namespace-Präfixe bei identischer Namespace-URI.

Fachlich relevante Unterschiede werden dagegen erkannt, beispielsweise:

- geänderte Elementwerte,
- fehlende Elemente,
- zusätzliche Elemente,
- geänderte Attribute,
- fehlende oder zusätzliche Attribute,
- unterschiedliche XML-Strukturen.

Dynamische Felder mit `ignore` ausschließen

SOAP-Responses enthalten häufig Werte, die sich bei jedem Aufruf ändern, beispielsweise:

- Zeitstempel,
- Request-IDs,
- Transaktions-IDs,
- technische UUIDs.

Solche Felder können über `ignore` vom Vergleich ausgeschlossen werden.

Beispiel:

```
1 - id: verify_patient_response
2   type: SOAP
3   op: ASSERT
4   response: patient_response
```

```

5   source: BODY
6   action: MATCH_REFERENCE
7
8   ignore:
9     - "//*[local-name()='timestamp']"
10    - "//*[local-name()='requestId']"
11    - "//*[local-name()='transactionId']"
12

```

Die angegebenen XPath-Ausdrücke werden sowohl auf die Referenz als auch auf die aktuelle Response angewendet.

Die entsprechenden XML-Knoten haben anschließend **keinen Einfluss auf das Vergleichsergebnis**.

Best Practice: `ignore` sollte nur für tatsächlich dynamische technische Werte verwendet werden. Fachlich relevante Daten sollten nicht ausgeschlossen werden, da sonst echte Regressionen unentdeckt bleiben können.

XML-Listen mit unterschiedlicher Reihenfolge

Bei vielen SOAP-Schnittstellen ist die Reihenfolge von Elementen innerhalb einer Liste nicht garantiert.

Beispielsweise können zwei fachlich identische Responses folgende Reihenfolgen liefern:

```

1  <persons>
2    <person>
3      <id>100</id>
4      <name>Anna</name>
5    </person>
6    <person>
7      <id>200</id>
8      <name>Peter</name>
9    </person>
10 </persons>
11

```

und:

```

1  <persons>
2    <person>
3      <id>200</id>
4      <name>Peter</name>
5    </person>
6    <person>
7      <id>100</id>
8      <name>Anna</name>
9    </person>
10 </persons>
11

```

Ein positionsabhängiger Vergleich würde hier fälschlicherweise eine Abweichung melden.

Mit `unordered` kann definiert werden, dass die Reihenfolge dieser Elemente keine Bedeutung besitzt:

```
1 - id: verify_patient_response
2   type: SOAP
3   op: ASSERT
4   response: patient_response
5   source: BODY
6   action: MATCH_REFERENCE
7
8   unordered:
9     - path: "//*[local-name()='persons']/*[local-name()='person']"
10       matchBy: "*[local-name()='id']"
11
```

`path` definiert die Elemente der Liste.

`matchBy` definiert einen stabilen Schlüssel innerhalb eines Listenelements, anhand dessen die Elemente einander zugeordnet werden.

Im Beispiel werden die `person`-Elemente anhand ihrer `id` verglichen. Die Reihenfolge innerhalb von `persons` spielt dadurch keine Rolle mehr.

Kombination von `ignore` und `unordered`

Beide Mechanismen können gemeinsam verwendet werden.

Beispiel:

```
1 - id: verify_patient_response
2   type: SOAP
3   op: ASSERT
4   response: patient_response
5   source: BODY
6   action: MATCH_REFERENCE
7
8   ignore:
9     - "//*[local-name()='timestamp']"
10    - "//*[local-name()='requestId']"
11
12   unordered:
13     - path: "//*[local-name()='persons']/*[local-name()='person']"
14       matchBy: "*[local-name()='id']"
15
```

Damit wird die vollständige SOAP-Response gegen die Referenz geprüft, wobei:

1. `timestamp` ignoriert wird,
 2. `requestId` ignoriert wird,
 3. die Reihenfolge der `person`-Elemente keine Rolle spielt,
 4. die Personen anhand ihrer `id` einander zugeordnet werden,
 5. alle übrigen fachlichen Inhalte weiterhin vollständig verglichen werden.
-

Referenzdateien

Die Referenzdateien werden automatisch relativ zum jeweiligen YAML-Testfall unterhalb eines `references` -Verzeichnisses abgelegt.

Das grundsätzliche Schema lautet:

```
1 <yaml-directory>/
2   └─ references/
3       └─ <yaml-name>/
4           └─ <testcase>__<step-id>.xml
5
```

Die `id` des ASSERT-Schritts sollte deshalb bewusst und stabil gewählt werden:

```
1 - id: verify_get_admin_response
2   type: SOAP
3   op: ASSERT
4   response: get_admin_response
5   source: BODY
6   action: MATCH_REFERENCE
7
```

Best Practice: Für `MATCH_REFERENCE` sollte immer eine explizite `id` vergeben werden. Dadurch bleibt der Name der Referenzdatei stabil, auch wenn davor weitere Testschritte eingefügt oder bestehende Schritte verschoben werden.

Die Referenzdateien sind Bestandteil des Tests und sollten gemeinsam mit den YAML-Testfällen versioniert werden.

Vollständiges Beispiel

```
1 steps:
2
3   # SOAP Request ausführen
4   - type: SOAP
5     op: EXEC
6     endpoint: "https://{E[env.ecard.service]}"
7     command: soap.foo.3.getadmininformationen_request_v11
8     response: get_admin_response
9     parameters:
10       dialogId: "{B[dialogId]}"
11       svNummer: "6457010521"
12
13   # HTTP Status prüfen
14   - type: SOAP
15     op: ASSERT
16     response: get_admin_response
17     source: STATUS
18     action: EQUALS
19     expected: "200"
20
21   # Komplette SOAP Response gegen Referenz prüfen
22   - id: verify_get_admin_response
```

```

23     type: SOAP
24     op: ASSERT
25     response: get_admin_response
26     source: BODY
27     action: MATCH_REFERENCE
28
29     ignore:
30       - "//*[local-name()='timestamp']"
31       - "//*[local-name()='requestId']"
32
33     unordered:
34       - path: "//*[local-name()='persons']/*[local-name()='person']"
35         matchBy: "*[local-name()='id']"
36

```

Damit kann mit einem einzigen ASSERT die vollständige fachliche SOAP-Antwort als Regressionstest abgesichert werden, ohne für jedes einzelne Response-Feld einen separaten XPath-ASSERT definieren zu müssen.

Zusammenfassung der Parameter

Parameter	Beschreibung	Erforderlich
<code>id</code>	Stabiler Bezeichner des ASSERT-Schritts und Bestandteil der Referenzzuordnung.	Empfohlen
<code>type</code>	Muss <code>SOAP</code> sein.	Ja
<code>op</code>	Muss <code>ASSERT</code> sein.	Ja
<code>response</code>	Name der zuvor durch einen SOAP-EXEC gespeicherten Response.	Ja
<code>source</code>	Für <code>MATCH_REFERENCE</code> wird der XML-Body mit <code>BODY</code> geprüft.	Ja
<code>action</code>	Muss <code>MATCH_REFERENCE</code> sein.	Ja
<code>ignore</code>	Liste von XPath-Ausdrücken für dynamische XML-Knoten, die vom Vergleich ausgeschlossen werden.	Nein

<code>unordered</code>	Liste von Regeln für XML-Elemente, deren Reihenfolge beim Vergleich keine Rolle spielt.	Nein
<code>unordered.path</code>	XPath auf die wiederholten Listenelemente.	Ja, wenn <code>unordered</code> verwendet wird
<code>unordered.matchBy</code>	Relativer XPath auf einen stabilen Schlüssel innerhalb des Listenelements.	Nein, aber empfohlen

Kurzform: `MATCH_REFERENCE` ist für den Vergleich einer **kompletten SOAP-Response** vorgesehen. Klassische ASSERTs mit `path`, `action` und `expected` bleiben weiterhin die bessere Wahl, wenn lediglich einzelne konkrete Werte überprüft werden sollen.

2.3. BUFFER (Datenextraktion)

Mit der `BUFFER`-Operation werden gezielt Daten mittels XPath aus einer gespeicherten SOAP-Antwort ausgelesen und in einer neuen Kontext-Variablen (`name`) hinterlegt.

Wichtige Regel: Verwende für `name` immer einen anderen Bezeichner als für `response`. Das Überschreiben eines originalen Response-Objekts mit einem String führt zu Laufzeitfehlern.

Unterstützte Quellen (`SOURCE`) und Verhalten:

Quelle (source)	Pfad (path)	Verhalten und Verarbeitung
<code>STATUS</code>	<i>Wird ignoriert</i>	Schreibt den numerischen HTTP-Statuscode der Antwort (z. B. <code>200</code> , <code>500</code>) in die Variable.
<code>HEADERS</code>	Header-Name	Sucht nach dem Wert eines bestimmten HTTP-Headers. Wenn

		der Header nicht existiert, wirft der Test einen Fehler.
BODY	XPath-Ausdruck	Führt eine XPath-Evaluierung auf dem XML-Body der Antwort aus und gibt den Textwert des Treffers zurück. Unterstützt auch die Filterung über <code>constraints</code> .

⚠ Wichtiger Hinweis zur BODY -Extraktion: Im Gegensatz zu einigen anderen Engines **ignoriert** die `SoapEngine` beim `BODY` den Parameter `action` komplett (Textaktionen wie `LINE`, `COUNT` oder `FILTER` werden nicht unterstützt). Das Ergebnis wird immer als Textwert (`String`) gespeichert, den der XPath-Ausdruck zurückgibt. Falls der XPath-Ausdruck keinen Treffer erzielt, wird der Wert `"NULL"` in die Variable geschrieben.

Konfigurationsbeispiele:

1. Extraktion eines Wertes aus dem XML-Body (via XPath):

```

1  - type: SOAP
2    op: BUFFER
3    name: dialogid
4    response: res_auth
5    source: BODY
6    path: "//dialogId"
```

1. Extraktion eines Wertes aus dem XML-Body (via XPath):

```

1  - name: "BrojUgovora"
2    op: "BUFFER"
3    response: "kreirajUgovorResponse"
4    source: "BODY"
5    path: "//*[local-name()='VertragId']/text()"
6
```

2. Speichern des HTTP-Statuscodes:

```

1  - name: "StvarniStatus"
2    op: "BUFFER"
3    response: "soapResponse"
4    source: "STATUS"
5
```

3. Extraktion eines Wertes aus einem HTTP-Header:

```

1 - name: "ServerTip"
2   op: "BUFFER"
3   response: "soapResponse"
4   source: "HEADERS"
5   path: "Server"

```

2.4. Constraints: Dynamische Ergebnisfilterung in XML-Listen

Der Zweck von Constraints ist die effiziente **Filterung von XML-Listen** und der vereinfachte Zugriff auf verschachtelte Datensätze. Wenn eine SOAP-Antwort Listen oder sich wiederholende Elemente enthält, erlauben Constraints das dynamische Auffinden des gewünschten Knotens anhand von Schlüssel-Wert-Kriterien, ohne dass komplexe, fest verdrahtete Index-XPaths verwendet werden müssen.

Struktur eines Constraints

Ein Constraint besteht aus folgenden Feldern:

Unter-Parameter	Beschreibung
path	Der relative oder absolute XPath zum Filter-Kriterium innerhalb des Listen-Elements.
action	Die Filter-Operation (z. B. <code>EQUALS</code>).
expected	Der erwartete Vergleichswert des Filter-Kriteriums.

Beispiel: Anwendung von Constraints

Im folgenden Beispiel wird eine Liste von Ländern abgerufen. Durch die `constraints` wird das Ergebnis automatisch so gefiltert, dass der `sISOCode` nur von dem Land extrahiert wird, dessen Spalte/Knoten `sName` exakt dem Wert **'Botswana'** entspricht.

```

1 - type: SOAP
2   op: BUFFER
3   name: isocode
4   response: response
5   source: BODY
6   path: "/*[local-name()='tCountryCodeAndNameGroupedByContinent']"
7   constraints:
8     - path: "/*[local-name()='tCountryCodeAndNameGroupedByContine
9       action: EQUALS
10      expected: 'Botswana'

```

Raw XML-Response des Servers

Der Zweck von Constraints ist die Filterung von Listen und der erleichterte Zugriff auf Daten innerhalb dieser Listen. Konkret hatten wir in diesem Fall eine Liste, und mithilfe von Constraints, die als Filter fungieren, können wir präzise auf die gewünschten Elemente der Liste zugreifen.

```
1 <<< BODY : <soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/
2 <<< BODY : <soap:Body>
3 <<< BODY : <m:ListOfCountryNamesGroupedByContinentRespons
4 <<< BODY : <m:ListOfCountryNamesGroupedByContinentRes
5 <<< BODY : <m:tCountryCodeAndNameGroupedByContine
6 <<< BODY : <m:Continent>
7 <<< BODY : <m:sCode>AF</m:sCode>
8 <<< BODY : <m:sName>Africa
9 <<< BODY : </m:Continent>
10 <<< BODY : <m:CountryCodeAndNames>
11 <<< BODY : <m:tCountryCodeAndName>
12 <<< BODY : <m:sISOCODE>DZ</m:sISOCODE
13 <<< BODY : <m:sName>Algeria</m:sName>
14 <<< BODY : </m:tCountryCodeAndName>
15 <<< BODY : <m:tCountryCodeAndName>
16 <<< BODY : <m:sISOCODE>AO</m:sISOCODE
17 <<< BODY : <m:sName>Angola</m:sName>
18 <<< BODY : </m:tCountryCodeAndName>
19 <<< BODY : <m:tCountryCodeAndName>
20 <<< BODY : <m:sISOCODE>BJ</m:sISOCODE
21 <<< BODY : <m:sName>Benin</m:sName>
22 <<< BODY : </m:tCountryCodeAndName>
23 <<< BODY : <m:tCountryCodeAndName>
24 ...
25 <<< BODY : <m:tCountryCodeAndName>
26 <<< BODY : <m:sISOCODE>WF</m:sISOCODE
27 <<< BODY : <m:sName>Wallis And Futuna
28 <<< BODY : </m:tCountryCodeAndName>
29 <<< BODY : <m:tCountryCodeAndName>
30 <<< BODY : <m:sISOCODE>WS</m:sISOCODE
31 <<< BODY : <m:sName>Western Samoa</m:
32 <<< BODY : </m:tCountryCodeAndName>
33 <<< BODY : </m:CountryCodeAndNames>
34 <<< BODY : </m:tCountryCodeAndNameGroupedByContin
35 <<< BODY : </m:ListOfCountryNamesGroupedByContinentRe
36 <<< BODY : </m:ListOfCountryNamesGroupedByContinentRespon
37 <<< BODY : </soap:Body>
38 <<< BODY : </soap:Envelope>
39 >>> SOAP STEP FINISHED | Status: SUCCESS | Duration: 276 ms
40
```

Log-Output der Engine

Die Engine dokumentiert den Filtervorgang transparent in den Logs:

```
1 >>> DSL - type: SOAP
2 >>> DSL op: BUFFER
3 >>> DSL name: isocode
4 >>> DSL response: response
5 >>> DSL source: BODY
6 >>> DSL path: "/*[local-name()='tCountryCodeAndNameGroupedByCont
7 >>> DSL constraints:
8 >>> DSL - path: "/*[local-name()='tCountryCodeAndNameGroupedBy
9 >>> DSL action: EQUALS
10 >>> DSL expected: 'Botswana'
11 >>> BUFFER : Value [BW] stored in variable [isocode]
12 >>> SOAP STEP FINISHED | Status: SUCCESS | Duration: 17 ms
13
```

Wichtige Vorteile der Constraints:

- **Präzise Datenextraktion:** Verhindert Fehler durch dynamische Zeilenverschiebungen in API-Antworten.
- **Saubere Testlogik:** Nachfolgende Schritte müssen sich nicht um Schleifen kümmern; die Engine liefert direkt den exakt passenden Wert.
- **Transparenz:** Da der Matching-Prozess und die extrahierte Variable direkt im Log ausgegeben werden, ist die Fehleranalyse sofort ersichtlich.

3. WS-Security & Authentifizierung

Die Engine unterstützt die automatische Generierung und Injektion von WS-Security (WSS) Headern direkt in den `<soap:Header>`. Die Konfiguration erfolgt deklarativ innerhalb der `metadata.json` des jeweiligen Kommandos.

Unterstützte Authentifizierungstypen

- `WSS_USERNAME_TOKEN_DIGEST`: Erstellt ein standardkonformes `wsse:UsernameToken` mit einem kryptografisch sicheren, transienten Nonce, dem aktuellen Zeitstempel (`wsu:Created`) und einem Password-Digest, berechnet als:

$$\\$\\text{\\{PasswordDigest\\}} = \\text{\\{Base64\\}}(\\text{\\{SHA-1\\}}(\\text{\\{Nonce\\}} + \\text{\\{Created\\}} + \\text{\\{Password\\}}))\\$\\$$$

Konfigurationsstruktur (`metadata.json`)

Die Authentifizierungsdaten werden über das optionale Objekt `"auth"` definiert:

```

1 {
2   "url" : "{{endpoint}}/EcSrvSimAbsAgent/serviceslbw/SOAPReceiverService",
3   "method" : "POST",
4   "headers" : {
5     "Content-Type" : "text/xml; charset=UTF-8",
6     "SOAPAction" : "\"\"",
7     "User-Agent" : "Agate-HttpClient (Version 1.0.0)"
8   },
9   "auth": {
10    "type": "WSS_USERNAME_TOKEN_DIGEST",
11    "username": "34ABS",
12    "password": "SECRET_PASSWORD"
13  }
14 }
15
```

3.1. Strukturierte Datei-Kopplung - Beispiel: `ProcessAbfrage`

Ein logisches Kommando (`command`) mappt immer direkt auf einen Ordner, der zwei Kernkomponenten enthält: die strukturellen Metadaten und das XML-Skelett.

3.2. Metadaten (metadata.json)

Definiert das Protokoll-Verhalten, HTTP-Header und Authentifizierungs-Anforderungen.JSON

```
1 {
2   "url" : "{{endpoint}}/services/SOAPReceiverService",
3   "method" : "POST",
4   "headers" : {
5     "Content-Type" : "text/xml;charset=UTF-8",
6     "SOAPAction" : "\"\""
7   },
8   "auth": {
9     "type": "WSS_USERNAME_TOKEN_DIGEST",
10    "username": "<username>",
11    "password": "<password>"
12  }
13 }
14 }
```

3.3. Request-Template (request.xml)

Enthält den reinen fachlichen SOAP-Body. **Wichtig:** Der `<soap:Header>` darf hier *nicht* manuell hinterlegt werden, wenn `auth` in den Metadaten aktiv ist. Die Engine injiziert den Header vollautomatisch vor dem Senden.XML

```
1 <soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
2   <soap:Body>
3     <ns4:ProcessAbfrageRequest xmlns="http://sozvers.at/ws/standard/v5/">
4       <ns4:StandardRequestHeader>
5         <QuellSystemApplikationsId>F00</QuellSystemApplikationsId>
6         <QuellSystemSystemId>B00</QuellSystemSystemId>
7         <KontextTransaktionsId>f71888ab-643e-46ef-8f98-4633e147a1fe</Ko
8         <KontextVerarbeitungsmodus>0</KontextVerarbeitungsmodus>
9       </ns4:StandardRequestHeader>
10      <ns4:SoftwareKennung>AGATE TS</ns4:SoftwareKennung>
11    </ns4:ProcessAbfrageRequest>
12  </soap:Body>
13 </soap:Envelope>
14
```

3.4. Request an Server

```
1 <soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
2   <soap:Header>
3     <wsse:Security soap:mustUnderstand="1" xmlns:wsse="http://docs.oasis-
4       <wsse:UsernameToken wsu:Id="UsernameToken-1fa8a39c-329b-472e-9116-8
5         <wsse:Username>34ABS</wsse:Username>
6         <wsse:Password Type="http://docs.oasis-open.org/wss/2004/01/oasis
7         <wsse:Nonce EncodingType="http://docs.oasis-open.org/wss/2004/01/
8         <wsu:Created>2026-06-27T16:29:55.667826900Z</wsu:Created>
9       </wsse:UsernameToken>
10    </wsse:Security>
11  </soap:Header>
12  <soap:Body>
13    <ns4:ProcessAbfrageRequest xmlns="http://sozvers.at/ws/standard/v5/">
14      <ns4:StandardRequestHeader>
15        <QuellSystemApplikationsId>F00</QuellSystemApplikationsId>
16        <QuellSystemSystemId>B00</QuellSystemSystemId>
17        <KontextTransaktionsId>f71888ab-643e-46ef-8f98-4633e147a1fe</Ko
18        <KontextVerarbeitungsmodus>0</KontextVerarbeitungsmodus>
19      </ns4:StandardRequestHeader>

```

```

20     <ns4:SoftwareKennung>AGATE TS</ns4:SoftwareKennung>
21     </ns4:ProcessAbfrageRequest>
22   </soap:Body>
23 </soap:Envelope>

```

4. Erweiterte Engine-Features & Unter-Prozesse

4.1. Dynamische Endpoint-Ersetzung

Falls das Feld `endpoint` im Testschritt definiert ist, wird das Vorkommen von `{{endpoint}}` in der Ziel-URL automatisch ersetzt. Die Engine entfernt hierbei intelligent einen eventuell vorhandenen abschließenden Schrägstrich (/) am Ende des Endpoints, um ungültige Doppel-Slashes (//) in der finalen URL zu verhindern.

4.2. Intelligente XML-Vorverarbeitung & Bereinigung

- **{EMPTY} -Handling:** Wird im XML-Body vor dem Senden automatisch durch einen leeren String ("") ersetzt.
- **{NULL} -Knoten-Bereinigung:** Alle XML-Elemente, deren Textinhalt exakt `{NULL}` entspricht, werden mitsamt ihren Kindknoten vollständig aus dem DOM-Baum gelöscht. Anschließend werden auch verwaiste, leere Tags (`<tag></tag>` oder `<tag/>`) sowie daraus resultierende Leerzeilen bereinigt. Dadurch wird sichergestellt, dass der Server nur valides XML ohne unerwünschte Leerfelder erhält.

4.3. Automatisches Antwort-Handling & Protokoll-Bereinigung

- **GZIP-Dekodierung:** Wenn die Serverantwort das Header-Attribut `Content-Encoding: gzip` enthält, erkennt die Engine dies automatisch und entpackt den Byte-Stream nahtlos in einen lesbaren UTF-8 String.
- **MTOM / Multipart-Bereinigung:** Enthält die API-Antwort einen komplexen Multipart-Container (z. B. bei binären MTOM-Anhängen), isoliert die Engine automatisch den reinen XML-Teil zwischen `<soap:Envelope` und `</soap:Envelope>`. Alle Transport-Header und Boundary-Inhalte des Multi-Parts werden vor der Speicherung im Kontext entfernt, um eine fehlerfreie XPath-Evaluierung zu garantieren.
- **Kontext-Speicherung:** Das finale, bereinigte Ergebnisobjekt (`RestResponse`) wird im Speicher unter dem in `response` definierten Schlüssel abgelegt und steht für nachfolgende Validierungen vollumfänglich zur Verfügung.

5. Referenzübersicht der Parameter nach Operationen

Schritt (op)	Parameter	Beschreibung	Erforderlich
--------------	-----------	--------------	--------------

EXEC (default)	endpoint	Basisadresse, die <code>{{endpoint}}</code> in der URL ersetzt.	Ja
	command	Der logische Pfad zum Verzeichnis, das die <code>metadata.json</code> und <code>request.xml</code> enthält.	Ja
	response	Name des Schlüssels für die Speicherung im Buffer.	Nein (Empfohlen)
	constraints	Liste von Bedingungen (<code>path</code> und <code>expected</code>) für komplexe XML-Listen.	Nein
ASSERT	response	Schlüssel, unter dem die SOAP-Antwort gespeichert wurde.	Ja
	source	Quelle für die Validierung: <code>STATUS</code> , <code>BODY</code> oder <code>HEADERS</code> .	Ja
	path	XPath-Ausdruck (für <code>BODY</code>) oder	Nein

		Name des HTTP-Headers.	
	action	Typ der Überprüfung (EQUALS , NOT_EQUALS , CONTAINS , NOT_CONTAINS , EMPTY , NOT_EMPTY).	Ja
	expected	Erwarteter Wert zum Abgleich (Zahl, Text oder {NULL}).	Nein
BUFFER	response	Schlüssel, unter dem die SOAP-Antwort gespeichert wurde.	Ja
	source	Quelle zum Lesen: STATUS , BODY oder HEADERS .	Ja
	path	XPath-Ausdruck zum Datenpunkt oder Header-Name.	Ja
	name	Name der Variable, in der der Wert gespeichert wird.	Ja
	constraints	Liste von Bedingungen (path und	Nein

		expected) für komplexe XML- Listen.	
--	--	--	--

6. Praxis-Tutorial: Kompletter SOAP-Lebenszyklus

Das folgende Beispiel zeigt ein typisches Integrationsszenario: Senden einer SOAP-Anfrage, Bereinigen nicht benötigter Felder über den `{NULL}` -Tag, Extrahieren einer ID aus einer XML-Liste mittels `constraints` und anschließendes Validieren des Status sowie des Inhalts.

Komplettes YAML-Szenario

1 | YAML

```
1  testCases:
2    - id: "TC_SOAP_WORKFLOW_DEMO"
3      description: "Kombinierter SOAP-Test: Request-Bereinigung, gefilterte
4        variables:
5          soap_host: "https://ws.example.local/v2"
6          searched_status: "PROCESSED"
7        steps:
8          # 1. Schritt: SOAP-Anfrage absenden mit dynamischer Knoten-Entfernung
9          - type: SOAP
10            url: "{{endpoint}}/OrderService"
11            endpoint: "{E[soap_host]}"
12            headers:
13              Content-Type: "text/xml; charset=UTF-8"
14              SOAPAction: "http://example.local/GetOrders"
15            body: |
16              <soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
17                <soapenv:Header/>
18                <soapenv:Body>
19                  <ord:GetOrdersRequest>
20                    <ord:FilterType>ALL</ord:FilterType>
21                    <ord:SecondaryFilter>{NULL}</ord:SecondaryFilter>
22                  </ord:GetOrdersRequest>
23                </soapenv:Body>
24              </soapenv:Envelope>
25            response: "order_res"
26
27          # 2. Schritt: Gezielte Extraktion aus einer XML-Liste unter Verweilzeit
28          # Die Engine sucht nach dem Element 'Order', wo der Status mit dem gesuchten übereinstimmt
29          - type: SOAP
30            op: BUFFER
31            response: "order_res"
32            source: "BODY"
33            path: "//*[local-name()='Order']/*[local-name()='Id']/text()"
34            name: "extracted_order_id"
35            constraints:
36              - path: "//*[local-name()='Order']/*[local-name()='Status']/text()"
37                expected: "{B[searched_status]}"
38
39          # 3. Schritt: Validierung des HTTP-Statuscodes der Antwort
40          - type: SOAP
41            op: ASSERT
42            response: "order_res"
43            source: "STATUS"
44            action: "EQUALS"
45            expected: "200"
46
```

```

47      # 4. Schritt: Direkte Inhaltsvalidierung im XML-Body mittels XPat
48      - type: SOAP
49        op: ASSERT
50        response: "order_res"
51        source: "BODY"
52        path: "//*[local-name()='ResponseStatus']/text()"
53        action: "EQUALS"
54        expected: "SUCCESS"
55

```

Darstellung der Konsolenausgabe (Log-Output)

Bei aktiviertem Verbose-Modus gibt der interne Logger der Engine die formatierten XML-Strukturen (*pretty-printed* mit 4 Leerzeichen Einzug) wie folgt aus:

```

1 | Plaintext
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35

```

```

>>> CALL      : POST https://ws.example.local/v2/OrderService (Initia
>>> CALL      : POST https://ws.example.local/v2/OrderService
>>> HEADER    : Content-Type: text/xml;charset=UTF-8
>>> HEADER    : SOAPAction: http://example.local/GetOrders
>>> BODY      : <soapenv:Envelope xmlns:soapenv="http://schemas.xmlso
>>> BODY      :     <soapenv:Header/>
>>> BODY      :     <soapenv:Body>
>>> BODY      :         <ord:GetOrdersRequest>
>>> BODY      :             <ord:FilterType>ALL</ord:FilterType>
>>> BODY      :         </ord:GetOrdersRequest>
>>> BODY      :     </soapenv:Body>
>>> BODY      : </soapenv:Envelope>
>>> BODY      :
<<< HTTP      : 200
<<< HEADER    : Content-Type: text/xml;charset=UTF-8
<<< BODY      : <soapenv:Envelope xmlns:soapenv="http://schemas.xmlso
<<< BODY      :     <soapenv:Body>
<<< BODY      :         <GetOrdersResponse>
<<< BODY      :             <ResponseStatus>SUCCESS</ResponseStatus>
<<< BODY      :             <Order>
<<< BODY      :                 <Id>ORD-99211</Id>
<<< BODY      :                 <Status>PENDING</Status>
<<< BODY      :             </Order>
<<< BODY      :             <Order>
<<< BODY      :                 <Id>ORD-55410</Id>
<<< BODY      :                 <Status>PROCESSED</Status>
<<< BODY      :             </Order>
<<< BODY      :         </GetOrdersResponse>
<<< BODY      :     </soapenv:Body>
<<< BODY      : </soapenv:Envelope>
<<< BODY      :
>>> BUFFER    : Value [ORD-55410] stored in variable [extracted_order
>>> SOAP ASSERT SUCCESS: EQUALS | Expected: [200], Actual: [200]
>>> SOAP ASSERT SUCCESS: EQUALS | Expected: [SUCCESS], Actual: [SUC

```

7. Troubleshooting & Häufige Fehler

Fehlermeldung: URL or Action not found for SOAP step

- **Ursache:** Weder das Feld `url` noch `action` wurden im YAML-Schritt definiert, oder das Mapping aus der vorgeschalteten Konfigurationsdatei (`metadata.json`) hat nicht gegriffen.

Unvollständige XML-Knoten im Log oder Sende-Body

- **Ursache:** Wenn Sie das Tag `{NULL}` innerhalb eines XML-Knotens übergeben (z. B. `<Knoten>{NULL}</Knoten>`), löscht die Methode `removeNullNodesFromXml` den gesamten Knoten aus dem Dokument.
- **Lösung:** Falls der Knoten strukturell zwingend als leerer Tag übertragen werden muss, nutzen Sie stattdessen das Tag `{EMPTY}`.

Der Filter-Buffer (`constraints`) liefert immer "NULL" zurück

- **Ursache:** Der Name des Listen-Elements wird im Java-Code über ein Splitting des Pfads ermittelt (`parts[parts.length - 2]`). Wenn Ihr XPath am Ende Funktionen wie `/text()` oder Attribute wie `/@id` enthält, verschiebt sich der Index.
- **Lösung:** Stellen Sie sicher, dass Ihr `path` für den gefilterten Buffer exakt nahtlos auf die Struktur `//Parent/Child/text()` abgestimmt ist, damit das vorletzte Element präzise dem Listenelement entspricht.

Sonderzeichen verfälschen das XML (`Failed to process XML`)

- **Ursache:** Unescaped Kaufmanns-Und-Zeichen (`&`) im Textkörper. Die Engine versucht zwar über eine Regex (`&(?!amp;|lt;...)`), freistehende `&` vorab in `&` zu wandeln; bei komplexen CDATA-Blöcken oder bereits teil-escaped Strings kann dies jedoch fehlschlagen und das DOM-Parsing blockieren.

Unterscheidung zwischen XPath mit und ohne `/text()` (Verhalten von `XPathConstants.STRING`)

Problem / Phänomen: Bei einfachen XML-Knoten liefern die Pfade `//*[local-name()='AddResult']` und `//*[local-name()='AddResult']/text()` oft exakt dasselbe Ergebnis (z. B. `"20"`). Wenn die XML-Struktur jedoch komplexer wird oder Kindknoten enthält, führt die Variante ohne `/text()` zu unerwarteten String-Verkettungen.

Ursache im Code: Die Engine evaluiert XPath-Ausdrücke intern über die Java-Schnittstelle als String:

```
1 | Java
1 | String result = (String) xpath.evaluate(query, doc, XPathConstants.STRING);
2 |
```

Gemäß der W3C-XPath-Spezifikation bewirkt `XPathConstants.STRING` bei einem selektierten **Element-Knoten** (Tag), dass der Textinhalt dieses Knotens *sowie aller seiner Kindknoten* rekursiv zusammengefügt wird.

Beispiel-Szenario mit komplexem XML:

Angenommen, der Server liefert folgende XML-Struktur:

```
1 | XML
```

```

1 <AddResult>
2   <Value>20</Value>
3   <Currency>EUR</Currency>
4 </AddResult>
5

```

Je nach XPath-Definition verhält sich die Engine beim **BUFFER** oder **ASSERT** völlig unterschiedlich:

XPath-Ausdruck	Selektiertes Objekt	Ergebnis der Engine (XPathConstants.STRING)
<code>//*[local-name()='AddResult']</code>	Kompletter Element-Knoten (<code><AddResult></code>)	"20EUR" (Der Text aller Kindknoten wird automatisch verkettet!)
<code>//*[local-name()='AddResult']/text()</code>	Direktes Text-Kind von <code><AddResult></code>	"NULL" (bzw. nur Whitespaces) (<code><AddResult></code> selbst hat keinen Text, nur Kindelemente)
<code>//*[local-name()='Value']/text()</code>	Text-Knoten von <code><Value></code>	"20" (Best Practice: Holt den exakten, isolierten Wert)

 **Entwickler-Tipp & Best Practice:** Nutze für einfache, skalare Werte (wie IDs oder einzelne Zahlen ohne Kindknoten) die kurze Variante ohne `/text()`. Sobald ein XML-Element jedoch verschachtelte Tags besitzt, **muss** der XPath präzise auf das innerste Element zeigen (z. B. `//*[local-name()='Value']/text()`), um zu verhindern, dass unerwünschte Strings oder Währungen in die Variable gemischt werden.